COMPSCI 713: AI Fundamentals

Lecture 9: Soft Computing

From Hard vs. Soft Computing to Fuzzy Logic and Naïve Bayes

Instructor: Xinyu Zhang
https://zhangxinyu-xyz.github.io/

School of Computer Science, University of Auckland

S1 2026

Overview

- Why do we need **Soft Computing**? What is the contrast with **Hard Computing**?
- A quick map of common soft computing techniques.
- **Fuzzy Logic**: reasoning with degrees of membership.
- **Naïve Bayes**: probabilistic classification with simplifying assumptions.
- A final comparison: *vagueness* vs. *uncertainty*.

Main message: soft computing does not aim for perfect certainty; it aims for useful decisions in messy worlds.

Why talk about "soft" computing at all?

- Many real-world questions are not cleanly true or false:
  - Is this person *tall*?
  - Is this patient *high risk*?
  - Is this email *spam*?
- Data may be incomplete, noisy, ambiguous, or only partly reliable.
- In such cases, strict symbolic rules can be brittle.

Goal: produce decisions that are *good enough*, robust, and interpretable.

"Is age > 65?"

"How spammy is this?"

"How risky is this?"

↓

Approximate reasoning

Hard Computing vs. Soft Computing

	Hard Computing	**Soft Computing**
Representation	crisp symbols, exact values	approximate values, degrees, probabilities
Logic	true / false	partial truth or belief
Typical setting	well-defined rules and precise inputs	noisy, incomplete, vague, uncertain inputs
Strength	exact answers when the model is right	robust behaviour when the world is messy
Examples	arithmetic, compilers, shortest path	fuzzy control, Bayes classifiers, neural nets

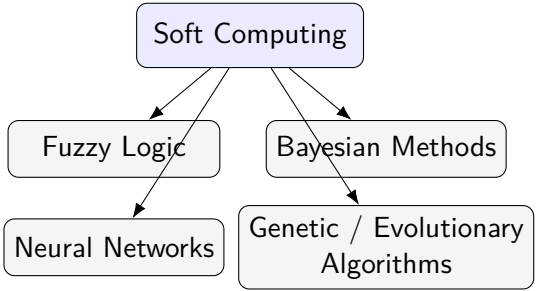
Important: "soft" does *not* mean weak. It means we tolerate imprecision in order to solve harder practical problems.

What is Soft Computing?

Working definition

A family of methods that explicitly deal with **uncertainty**, **partial truth**, **inaccuracy**, and **approximation**.

- Historically contrasted with early AI's emphasis on crisp logic and theorem proving.
- Especially useful when:
  - the environment is noisy,
  - expert knowledge is vague,
  - or a precise mathematical model is unavailable.



Two different issues: uncertainty vs. vagueness

Probabilistic uncertainty

We do not know which state of the world is true.

$$P(\text{disease} \mid \text{symptoms})$$

- concerns **belief** under incomplete knowledge
- natural tool: **Bayesian reasoning**

Fuzzy vagueness

The concept itself has blurry boundaries.

$$\mu_{\text{tall}}(183 \text{ cm}) = 0.6$$

- concerns **degree of membership**
- natural tool: **Fuzzy logic**

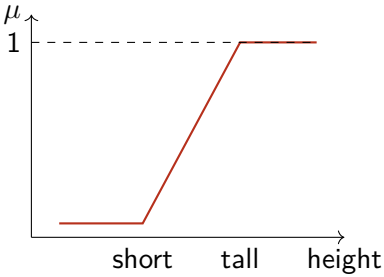
Key distinction: probability asks *"how likely?"*; fuzzy logic asks *"to what degree?"*

Soft Computing in this lecture

- Fuzzy Logic
- Bayesian Reasoning

Fuzzy Set Theory: from crisp membership to graded membership

- In classical set theory, an element either belongs to a set or it does not.
- In fuzzy set theory, membership is a value in $[0, 1]$.
- A **membership function** $\mu_A(x)$ describes how **strongly** x belongs to set A.



Example

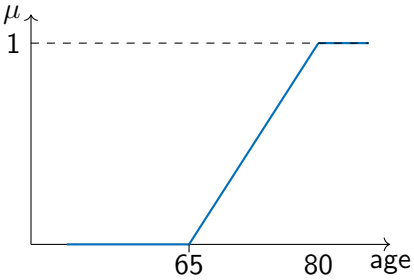
For the fuzzy set *Tall*, a person of height 183cm might have

$$\mu_{\text{Tall}}(183) = 0.6$$

This is **not** a 60% chance of being tall; it is a 0.6 degree of membership in the concept *Tall*.

Example: "Over 65" is crisp, but care recommendations are not

- A rule such as "vaccinate people over 65" is crisp.
- But clinically, the recommendation becomes stronger *gradually*, not suddenly.
- Fuzzification captures this transition more naturally than a hard threshold.



Interpretation:

- 60 years old: recommendation is weak
- 65 years old: moderate
- 80 years old: strong

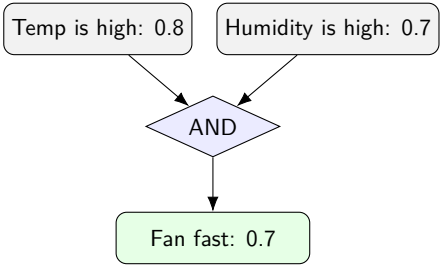
This is often more faithful to expert judgement than a sharp cut-off.

Fuzzy Logic: combining factors and firing rules

Common ingredients

- fuzzy variables and membership functions
- aggregation operators such as AND / OR
- rules of the form:

$$\text{IF } x \text{ AND } y \text{ THEN } z$$



Simple example

- IF temperature is *high* AND humidity is *high* THEN fan speed is *fast*
- Suppose $\mu_{\text{high temp}} = 0.8$ and $\mu_{\text{high humidity}} = 0.7$
- Using a common fuzzy AND, the rule fires with strength $\min(0.8, 0.7) = 0.7$

Fuzzy Logic

- A logical system for reasoning with **fuzzy variables**, where truth can take values in $[0, 1]$ rather than only $\{\text{true, false}\}$.
- Common fuzzy connectives can be defined as:

$$A \vee B = \max(A, B)$$

$$A \wedge B = \min(A, B)$$

$$\neg A = 1 - A$$

- These operators generalise familiar Boolean logic to graded truth values.
- We can also define **truth tables** for fuzzy logic, but the entries are no longer restricted to only 0 and 1.

A	B	$A \vee B$
0	0	0
0	0.5	0.5
0	1	1
0.5	0	0.5
0.5	0.5	0.5
0.5	1	1
1	0	1
1	0.5	1
1	1	1

Example truth table for fuzzy OR

Fuzzy Inference

- Inference in fuzzy logic is more complex than in classical logic.
- Recall that in classical logic:

$$A \to B \Leftrightarrow \neg A \vee B$$

- We can extend this idea to fuzzy logic:

$$A \to B = \max(1 - A, B)$$

- This defines a fuzzy implication operator.
- However, the resulting truth table may seem unintuitive.
- For example:

$$0.5 \to 0 = 0.5$$

whereas we might expect it to be closer to 0.

A	B	$A \to B$
0	0	1
0	0.5	1
0	1	1
0.5	0	0.5
0.5	0.5	0.5
0.5	1	1
1	0	0
1	0.5	0.5
1	1	1

Example truth table for fuzzy implication

Fuzzy Inference (2)

- An alternative is the **Gödel implication**, defined as:

$$A \to B = \begin{cases} 1 & \text{if } A \leq B \\ B & \text{otherwise} \end{cases}$$

- This can also be written informally as:

$$A \to B = (A \leq B) \ \vee \ B$$

- Compared to the previous definition, this gives a more intuitive behaviour. For example:

$$0.5 \to 0 = 0$$

- This aligns better with our intuition that *"if A holds but B does not, the implication should be false."*

A	B	$A \to B$
0	0	1
0	0.5	1
0	1	1
0.5	0	0
0.5	0.5	1
0.5	1	1
1	0	0
1	0.5	0.5
1	1	1

Example truth table for Gödel implication

Where has Fuzzy Logic worked especially well?

- **Control systems**: Autopilot (manned and unmanned planes, ships), automatic transmissions, anti-lock braking systems (ABS)
- **Consumer devices**: washing machines, vacuum cleaners, refrigerators (temperature, power, humidity)
- **Decision support**: combining expert rules with imprecise concepts

Why useful in control?

- can encode expert heuristics directly
- does not require an exact mathematical model
- behaves smoothly rather than flipping at hard thresholds

Application: Fuzzy Control

Not unique to FL, in control systems it can be useful to give inputs not just on error, but also on rate of change:

Typical control inputs

$$\text{error } e(t)$$

$$\Delta e(t) = e(t) - e(t-1)$$

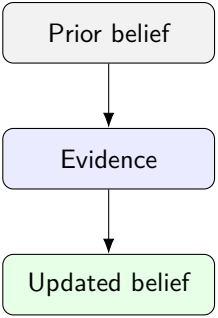
E.g., A controller may react differently when we are below target but moving *towards* the target than when we are below target and drifting *away*.

Soft Computing in this lecture

- Fuzzy Logic
- Bayesian Reasoning

Bayesian reasoning: updating belief with evidence

- Bayesian reasoning is about **uncertainty**: which explanation is most plausible given the evidence?
- Start with a **prior** belief.
- Observe evidence.
- Compute the **posterior**: updated belief after seeing the evidence.



Bayes' theorem

$$P(H \mid e) = \frac{P(e \mid H)P(H)}{P(e)}$$

- $P(H)$: **prior probability** of hypothesis H (belief in hypothesis H before seeing the evidence)
- $P(e \mid H)$: **likelihood** of evidence under H (how likely the evidence e is if H is true)
- $P(H \mid e)$: **posterior probability** after seeing e (updated belief in H after observing e)

- **Key idea:** posterior $\propto$ prior $\times$ likelihood

A classic Bayes example: alarm $\neq$ burglary

Your burglar alarm wakes you one night. What's your rational belief that a burglary attempt has taken place in your home?

Suppose:

- The alarm detects burglary with probability 0.95
- False alarm rate is 0.01
  - Based on previous false alarms, there's a 1% chance that your alarm will go off for some other reason at some point on a given night (e.g. thunder, cat)
- Burglary rate is 0.0001 per night
  - Previous crime patterns for the area indicate one house in 10,000 will be burgled on a given night

A classic Bayes example: alarm $\neq$ burglary

- Let: $H =$ burglary, $\quad e =$ alarm sounding
- Given: $P(e \mid H) = 0.95, \quad P(H) = 0.0001,$ $P(e \mid \neg H) = 0.01$
- Total probability:

$$P(e) = P(e \mid H)P(H) + P(e \mid \neg H)P(\neg H)$$

$$= 0.95 \times 0.0001 + 0.01 \times 0.9999 = 0.010094$$

- Bayes Rule:

$$P(H \mid e) = \frac{P(e \mid H)P(H)}{P(e)} = \frac{0.95 \times 0.0001}{0.010094} \approx 0.0094$$

Interpretation

- The probability of a burglary is still **less than 1%**.
- So, don't overreact — the alarm is not definitive evidence.
- However, it is still **strong evidence**:
  - It increases the probability by almost 100×
- Evidence updates belief:

e influences our belief in H

Naïve Bayes classifier

- We want to assign an input $x = (x_1, \ldots, x_n)$ to a class $c \in C$.
- Bayes tells us:

$$P(C = c \mid x) = \frac{P(C = c)P(x \mid C = c)}{P(x)}$$

- The **naïve** assumption is that features are **conditionally independent given the class**:

$$P(x \mid C = c) \approx \prod_i P(x_i \mid C = c)$$

  - $c \in C$: the **class label**, where C is the set of possible classes (e.g., *spam* / *not spam*, *disease* / *no disease*)
  - $x = (x_1, \ldots, x_n)$: a **feature vector** describing one instance
  - x_i: the i-th **feature** (e.g., whether a word appears in an email, or a measurable attribute, such as whether the app attempts to change file permissions)

Naïve Bayes: why it works despite unrealistic assumptions

Assumptions

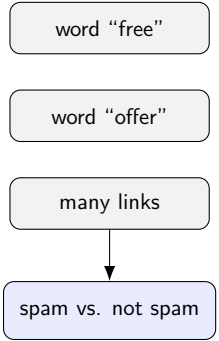
- features are independent given the class
- each feature contributes multiplicatively to the class score

Reality

- features are almost never truly independent
- some features matter much more than others. E.g., "FREE!!!" is stronger than "hello"

Still, it often works because

- classification may only need the *ranking* of classes. E.g., $P(spam|x) = 0.3$; $P(notspam|x) = 0.2$
- many weak signals combine effectively
- parameter estimation is easy even with limited data. No assumption (2^n) vs. Naïve Bayes (n)



Example application: spam detection

- For spam detection, let:

$$H \in \{\text{spam},\ \text{not spam}\}$$

  denote the hypothesis (class label), and let

$$e = (e_1, e_2, \ldots, e_n)$$

  denote the evidence extracted from an email.
- Each e_i can represent a feature such as:
  - whether the word "free" appears,
  - whether the word "offer" appears,
  - whether the email contains many links.
- Using Naïve Bayes, we choose the hypothesis with the largest posterior:

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} P(H \mid e)$$

Example application: spam detection (cont'd)

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} P(H \mid e)$$

- By Bayes' theorem:

$$P(H \mid e) \propto P(H)\, P(e \mid H)$$

- With the naïve conditional independence assumption:

$$P(e \mid H) \approx \prod_{i=1}^{n} P(e_i \mid H)$$

- Therefore:

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} P(H) \prod_{i=1}^{n} P(e_i \mid H)$$

Example application: spam detection (cont'd)

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} P(H) \prod_{i=1}^{n} P(e_i \mid H)$$

- Taking **logs** gives an equivalent scoring rule:

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} \left[\log P(H) + \sum_{i=1}^{n} \log P(e_i \mid H) \right]$$

- Interpretation:
  - $\log P(H)$ is the prior contribution
  - each $\log P(e_i \mid H)$ is the contribution of one feature
  - evidence accumulates additively in log-space
- So, words that are **much more common** in spam than in non-spam increase the score for $H = \text{spam}$.

Example application: spam detection (cont'd)

$$\hat{H} = \text{argmax}_{H \in \{\text{spam,not spam}\}} \left[\log P(H) + \sum_{i=1}^{n} \log P(e_i \mid H) \right]$$

Log-score avoids numerical underflow and makes each feature's contribution easier to interpret.

Strengths:

- simple
- fast
- scalable
- strong baseline for text tasks

Additional material: `https://www.ibm.com/think/topics/naive-bayes`

Fuzzy Logic vs. Naïve Bayes

	Fuzzy Logic	**Naïve Bayes**
Core idea	degree of membership / partial truth	probability of a class given evidence
Main issue handled	vagueness in concepts	uncertainty in hypotheses
Typical input	expert rules, linguistic variables	labelled data and feature counts
Typical output	control action, recommendation strength	class label or posterior score
Best suited for	smooth rule-based decision/control	lightweight probabilistic classification
Question answered	"To what degree is this true?"	"How likely is this class?"

Takeaway: both are **Soft Computing**, but they model different kinds of imperfection.

Summary

- **Soft Computing** addresses problems involving uncertainty, imprecision, and approximation.
- **Hard Computing** prefers exact symbols and crisp logic; **Soft Computing** tolerates ambiguity to gain practical usefulness.
- **Fuzzy Logic** models *graded concepts* through membership functions and fuzzy rules.
- **Naïve Bayes** models *uncertain classification* using Bayes' theorem and an independence assumption.
- These two methods are different, but together they show the central idea of soft computing:

One-sentence conclusion

In an uncertain world, we often do not need a perfect answer — we need a useful, robust, and explainable one.

Quick Quiz

Q1. (Concept)

- Which statement is correct?
  - A. Fuzzy logic models uncertainty about which state is true
  - B. Bayesian reasoning models degree of membership
  - C. Fuzzy logic models vagueness in concepts

Quick Quiz

Q2. (Fuzzy Logic)

- Suppose $\mu_A = 0.6$ and $\mu_B = 0.8$. Using fuzzy AND:

$$A \wedge B = ?$$

A. 0.6
B. 0.8
C. 0.48

Quick Quiz

Q3. (Naïve Bayes)

- What is the key assumption of Naïve Bayes?
  - A. Features are equally important
  - B. Features are conditionally independent given the class
  - C. Classes are independent

Thank you!

Q&A